

LAB 7 – HASHING (1230519)

1. Hashing is a technique used for performing insertions, deletions and find in constant average time
2. – easy to compute
 - Approximates a random function : for every input, every output is equally likely (simple uniform hashing)
3. Fulfil good hash function and there will be no collisions
4. Two or more keys hash to the same slot. Collision resolution techniques which are separate chaining and open addressing such as bucket addressing, linear probing, quadratic probing and double hashing
- 5.

5. 19, 50, 89, 40

$h(k) = K \% 5$

0	50
1	89
2	40
3	
4	19

$19 \% 5 = 4$
 $50 \% 5 = 0$
 $89 \% 5 = 4$
 $40 \% 5 = 0$

6. data = 4, 14, 19, 9004, 33

$h(k) = k \% 5$

0	14
1	19
2	9004
3	
4	4

$4 \% 5 = 4$
 $14 \% 5 = 4$
 $19 \% 5 = 4$
 $9004 \% 5 = 4$
 $33 \% 5 = 3$

6.

7.	11, 33, 36, 16, 55, 69, 49, 82, 66, 5	$h(k) = k \% 7$		
	0 49		$11 \% 7 = 4$	
	1 3		$33 \% 7 = 5$	
	2 16		$36 \% 7 = 1$	
	3 66		$16 \% 7 = 2$	
	4 11		$55 \% 7 = 6$	
	5 33, 82, 5		$69 \% 7 = 6$	
	6 55, 69		$49 \% 7 = 0$	
			$82 \% 7 = 5$	
			$66 \% 7 = 3$	
			$5 \% 7 = 5$	
8.	data = 4, 12, 8, 3, 10	$h(k) = k \% 4$		
	0 4, 12, 8		$4 \% 4 = 0$	
	1		$12 \% 4 = 0$	
	2 10		$8 \% 4 = 0$	
	3 3		$3 \% 4 = 3$	
			$10 \% 4 = 2$	
9.	19, 50, 89, 39	$h(k) = k \% 4, h_2 = 3 - (x \% 3)$		
	0 39		$19 \% 4 = 3$	
	1 89		$50 \% 4 = 2$	
	2 50		$89 \% 4 = 1$	
	3 19		$39 \% 4 = 3$	

7.

10.	data = 4, 9, 14, 1, 19			
	0 14	$4 \% 5 = 4$		
	1 1	$9 \% 5 = 4$	$3 - (9 \% 3) = 3$	
	2 9	$14 \% 5 = 4$	$3 - (14 \% 3) = 1$	
	3 19	$1 \% 5 = 1$		
	4 4	$19 \% 5 = 4$	$3 - (19 \% 3) = 2$	
11.	data = 89, 18, 49, 58, 62, 6, 60	$h(k) = k \% 10, h_2 = 10 - (x \% 10)$		
	0 49	$89 \% 10 = 9$		
	1	$18 \% 10 = 8$	$10 - (49 \% 10) = 1$	
	2 58	$49 \% 10 = 9$	$10 - (58 \% 10) = 2$	
	3	$58 \% 10 = 8$	$10 - (62 \% 10) = 8$	
	4 6	$62 \% 10 = 2$	$10 - (6 \% 10) = 4$	
	5	$6 \% 10 = 6$	$10 - (60 \% 10) = 10$	
	6 62	$60 \% 10 = 0$		
	7			
	8 18			
	9 89			

PROGRAMMING TASK

Linear Probing

```
import java.io.*;
import java.util.*;
import java.util.Scanner;

class LinearProbingHashTable {

    private int currentSize, maxSize;
    private String[] keys;
    private String[] vals;

    public LinearProbingHashTable(int capacity)
    {
        currentSize = 0;
        maxSize = capacity;
        keys = new String[maxSize];
        vals = new String[maxSize];
    }

    public void makeEmpty()
    {
        currentSize = 0;
        keys = new String[maxSize];
        vals = new String[maxSize];
    }

    public int getSize() { return currentSize; }

    public boolean isFull()
    {
        return currentSize == maxSize;
    }

    public boolean isEmpty() { return getSize() == 0; }

    public boolean contains(String key)
    {
        return get(key) != null;
    }

    private int hash(String key)
    {
        return key.hashCode() % maxSize;
    }
}
```

```

public void insert(String key, String val)
{
    int tmp = hash(key);
    int i = tmp;

    do {
        if (keys[i] == null) {
            keys[i] = key;
            vals[i] = val;
            currentSize++;
            return;
        }

        if (keys[i].equals(anObject: key)) {
            vals[i] = val;
            return;
        }

        i = (i + 1) % maxSize;
    }

```

```

        while (i != tmp);
    }

    public String get(String key)
    {
        int i = hash(key);
        while (keys[i] != null) {
            if (keys[i].equals(anObject: key))
                return vals[i];
            i = (i + 1) % maxSize;
        }
        return null;
    }

    public void remove(String key)
    {
        if (!contains(key))
            return;

        int i = hash(key);
        while (!key.equals(keys[i]))
            i = (i + 1) % maxSize;

```

```

        keys[i] = vals[i] = null;

        for (i = (i + 1) % maxSize; keys[i] != null;
             i = (i + 1) % maxSize) {
            String tmp1 = keys[i], tmp2 = vals[i];
            keys[i] = vals[i] = null;
            currentSize--;
            insert(key:tmp1, val:tmp2);
        }
        currentSize--;
    }

    public void printHashTable()
    {
        System.out.println(x: "\nHash Table: ");
        for (int i = 0; i < maxSize; i++)
            if (keys[i] != null)
                System.out.println(keys[i] + " " + vals[i]);
        System.out.println();
    }
}

public class GFG {

    public static void main(String[] args)
    {

        Scanner scan = new Scanner(source: System.in);

        System.out.println(x: "Hash Table Test\n\n");
        System.out.println(x: "Enter size");

        LinearProbingHashTable lpht
            = new LinearProbingHashTable(capacity: scan.nextInt());

        char ch;

        do

        {

            System.out.println(x: "\nHash Table Operations\n");
            System.out.println(x: "1. insert ");

```

```
System.out.println(x: "1. insert ");
System.out.println(x: "2. remove");
System.out.println(x: "3. get");
System.out.println(x: "4. clear");
System.out.println(x: "5. size");
```

```
int choice = scan.nextInt();
```

```
switch (choice) {
```

```
case 1:
```

```
    System.out.println(x: "Enter key and value");
    lpht.insert(key:scan.next(), val:scan.next());
```

```
    break;
```

```
case 2:
```

```
    System.out.println(x: "Enter key");
    lpht.remove(key:scan.next());
```

```
    break;
```

```
case 3:
```

```
    System.out.println(x: "Enter key");
    System.out.println("Value = "
                       + lpht.get(key:scan.next()));
```

```
    break;
```

```
case 4:
```

```
    lpht.makeEmpty();
    System.out.println(x: "Hash Table Cleared\n");
```

```
    break;
```

```
case 5:
```

```
    System.out.println("Size = "
                       + lpht.getSize());
    break;
```

```
default:
```

```
    System.out.println(x: "Wrong Entry \n ");
    break;
```

```
}
```

```
lpht.printHashTable();
```

```
System.out.println(
    x: "\nDo you want to continue (Type y or n) \n");
```

```
    ch = scan.next().charAt(index: 0);
} while (ch == 'Y' || ch == 'y');
```

Hash Table Operations

1. insert
2. remove
3. get
4. clear
5. size

1

Enter key and value

F 121

Hash Table:

F 121

Do you want to continue (Type y or n)

Y

Hash Table Operations

1. insert
2. remove
3. get
4. clear
5. size

1

Separate chaining

```
import java.util.ArrayList;
import java.util.Objects;

class HashNode<K, V> {
    K key;
    V value;
    final int hashCode;

    HashNode<K, V> next;

    public HashNode(K key, V value, int hashCode)
    {
        this.key = key;
        this.value = value;
        this.hashCode = hashCode;
    }
}

class Map<K, V> {
    private ArrayList<HashNode<K, V> > bucketArray;
```



```

private int numBuckets;
private int size;
public Map()
{
    bucketArray = new ArrayList<>();
    numBuckets = 10;
    size = 0;

    for (int i = 0; i < numBuckets; i++)
        bucketArray.add(e: null);
}

public int size() { return size; }
public boolean isEmpty() { return size() == 0; }

private final int hashCode (K key) {
    return Objects.hashCode(o: key);
}

private int getBucketIndex(K key)
{
    int hashCode = hashCode(key);
    int index = hashCode % numBuckets;

    index = index < 0 ? index * -1 : index;
    return index;
}

public V remove(K key)
{
    int bucketIndex = getBucketIndex(key);
    int hashCode = hashCode(key);

    HashNode<K, V> head = bucketArray.get(index: bucketIndex);

    HashNode<K, V> prev = null;
    while (head != null) {
        if (head.key.equals(obj: key) && hashCode == head.hashCode)
            break;

        prev = head;
        head = head.next;
    }
}

```

```

        if (head == null)
            return null;

        size--;

        if (prev != null)
            prev.next = head.next;
        else
            bucketArray.set(index: bucketIndex, element: head.next);

        return head.value;
    }

```

```

public V get(K key)
{
    int bucketIndex = getBucketIndex(key);
    int hashCode = hashCode(key);

    HashNode<K, V> head = bucketArray.get(index: bucketIndex);

```

```

    while (head != null) {
        if (head.key.equals(obj:key) && head.hashCode == hashCode)
            return head.value;
        head = head.next;
    }

    return null;
}

```

```

public void add(K key, V value)
{
    int bucketIndex = getBucketIndex(key);
    int hashCode = hashCode(key);
    HashNode<K, V> head = bucketArray.get(index: bucketIndex);

    while (head != null) {
        if (head.key.equals(obj:key) && head.hashCode == hashCode) {
            head.value = value;
            return;
        }
        head = head.next;
    }

```

```

        size++;
        head = bucketArray.get(index: bucketIndex);
        HashNode<K, V> newNode
            = new HashNode<K, V>(key, value, hashCode);
        newNode.next = head;
        bucketArray.set(index: bucketIndex, element: newNode);
    }

    if ((1.0 * size) / numBuckets >= 0.7) {
        ArrayList<HashNode<K, V> > temp = bucketArray;
        bucketArray = new ArrayList<>();
        numBuckets = 2 * numBuckets;
        size = 0;
        for (int i = 0; i < numBuckets; i++)
            bucketArray.add(e: null);

        for (HashNode<K, V> headNode : temp) {
            while (headNode != null) {
                add(key: headNode.key, value: headNode.value);
                headNode = headNode.next;
            }
        }
    }
}

```

Double Hashing Question 10

```

import java.util.Arrays;

public class DoubleHashing {
    private static final int TABLE_SIZE = 5;
    private int[] hashTable;
    private static final int EMPTY = -1;

    public DoubleHashing() {
        hashTable = new int[TABLE_SIZE];
        Arrays.fill(a: hashTable, val: EMPTY);
    }

    private int primaryHash(int key) {
        return key % TABLE_SIZE;
    }

    private int secondaryHash(int key) {
        return 3 - (key % 3);
    }

    public void insert(int key) {

```

```

int hash1 = primaryHash(key);
int hash2 = secondaryHash(key);

int index = hash1;
int i = 0;

while (hashTable[index] != EMPTY) {
    i++;
    index = (hash1 + i * hash2) % TABLE_SIZE;
}

hashTable[index] = key;
}

public void display() {
    System.out.println("Hash Table:");
    for (int i = 0; i < TABLE_SIZE; i++) {
        System.out.println("Index " + i + ": " + (hashTable[i] == EMPTY ? "EMPTY" : hashTable[i]));
    }
}

public static void main(String[] args) {
    DoubleHashing dh = new DoubleHashing();

    public static void main(String[] args) {
        DoubleHashing dh = new DoubleHashing();

        int[] data = {4, 9, 14, 1, 19};

        for (int key : data) {
            dh.insert(key);
        }

        dh.display();
    }
}

```

```

] --- exec:3.1.0:ex
Hash Table:
Index 0: 14
Index 1: 1
Index 2: 9
Index 3: 19
Index 4: 4
-----

```

Quadratic probing

```
class GFG2 {  
  
    static void printArray(int arr[])  
    {  
        for (int i = 0; i < arr.length; i++) {  
            System.out.print(arr[i] + " ");  
        }  
    }  
  
    static void hashing(int table[], int arr[])  
    {  
        int tsize = table.length, n = arr.length;  
  
        for (int i = 0; i < n; i++) {  
  
            int hv = arr[i] % tsize;  
  
            if (table[hv] == -1)  
                table[hv] = arr[i];  
  
            else {  
  
                for (int j = 1; j <= tsize; j++) {  
  
                    int t = (hv + j * j) % tsize;  
                    if (table[t] == -1) {  
                        table[t] = arr[i];  
                        break;  
                    }  
                }  
            }  
        }  
  
        printArray(arr:table);  
    }  
  
    public static void main(String args[])  
    {  
        int arr[] = { 3,5,13,24,33,45,54 };  
  
        int tsize = 11;  
        int hash_table[] = new int[tsize];  
    }  
}
```

```
    for (int i = 0; i < tsize; i++) {  
        hash_table[i] = -1;  
    }  
    hashing(table: hash_table, arr);  
}
```

```
--- exec:3.1.0:exec (default-cli) @ palindrome2 ---  
33 45 13 3 -1 5 24 -1 -1 -1 54
```

```
-----  
BUILD SUCCESS  
-----
```